

BOH Ransomware Writeup

22nd September 2025

Prepared By: Hybread

Challenge Author: Hybread

Difficulty: Hard

Writeup classification: Official

Table of Contents

Table of Contents	1
1 – Introduction	1
2 – Description	2
3 – Skills Required	2
4 – Skills Learnt	2
5 – Solve	2

1 – Introduction

BOH Ransomware is a hard reversing challenge. The executable file provided to the user includes a malware-like trait which has anti-debugging code and anti-antivirus checks implemented into it to prevent dynamic analysis. To solve, the user will have to reverse engineer the provided 'ransomware' windows executable, written in Nim. By analyzing the code & embedded virtual machine, they can create a script to emulate the VM to execute bytecode stored inside injector.bin to retrieve the AES key. After obtaining the key, the user has to understand how the IV and Ciphertext is stored inside the encrypted image, then the user can write a python or Nim script to decrypt MasterKey.png.BoH using AES-CTR.

2 – Description

You return to your workstation after a long day out in BoH Cyber City. A note is attached which has the following:

"Hey, I noticed you dropped your master key while you were out and I just so happened to pick it up. What timing! Anyways, if you want it back~~ how's \$100,000 sound? ;)"

- EvilBread

3 – Skills Required

- Advanced reversing knowledge
- Advanced decompiled code analysis
- Familiar with hashing functions
- Knowledge of executable packers such as UPX
- Familiar with Assembly language
- Familiar with virtual machine obfuscation

4 – Skills Learnt

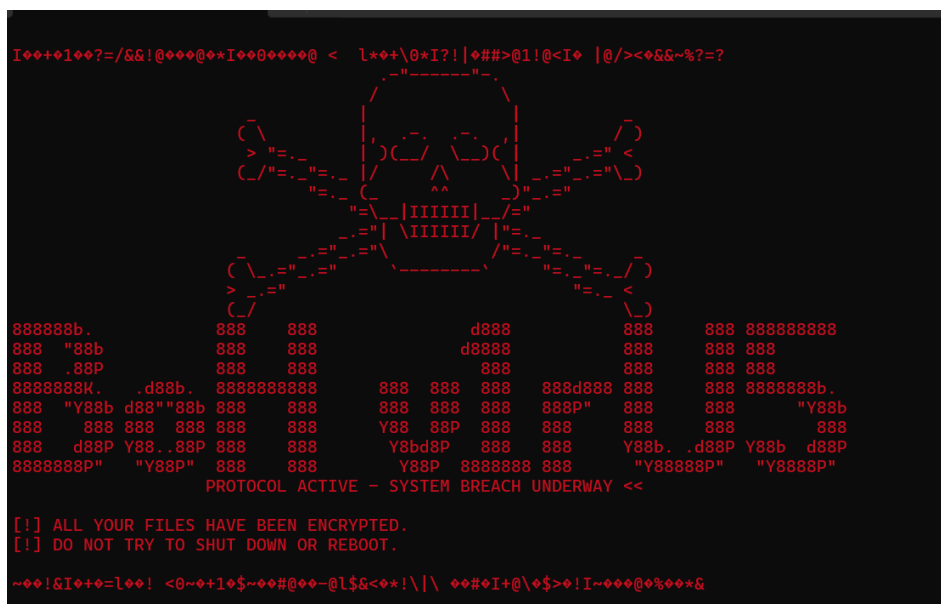
- Learn how to unpack UPX-packed executables
- Learn how to write emulators for virtual machines
- Learn about Nim language and its library usage
- Learn how to script in Nim/Python for cryptographic tasks
- Learn about Fowler-Noll-Vo (FNV) and Rolling Hash hashing functions
- Learn about AES-CTR mode and it's encryption method

5 – Solve

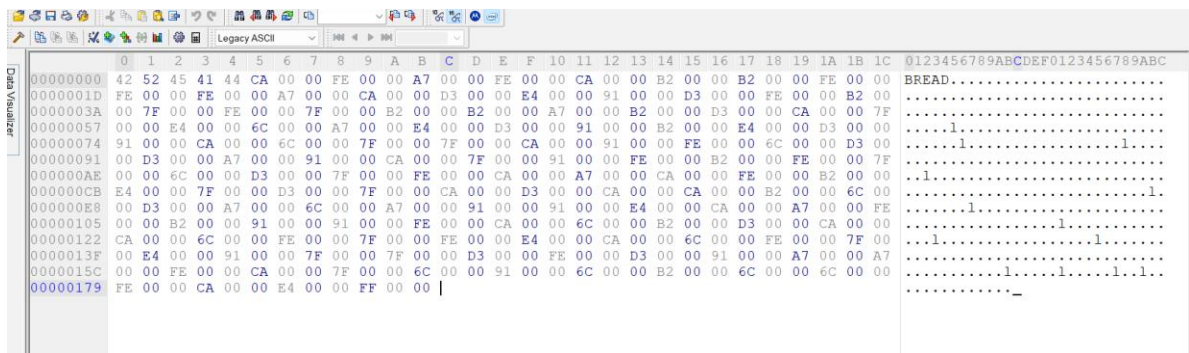
Downloading and unzipping the contents within the .zip file, we're greeted with these 3 files: A windows .exe file, a binary file and a corrupted .png file

 BlackShell.exe	18/7/2025 15:59	Application	219 KB
 injector.bin	18/7/2025 11:11	BIN File	1 KB
 MasterKey.png.BoH	18/7/2025 11:13	BOH File	1,996 KB

If the user tries to run the BlackShell.exe, they will be greeted with a malware-like message in their terminal:



If the user heads back to the files, they will find out that the program itself is missing as it is made to detect any form of antivirus software like Microsoft Defender, and if detected, it will self-wipe. Anyways, if analyzing the .png file and injector.bin respectively, the user can find these in any hex-viewer.



Based on how the bytes are written, an experienced user can assume that it the binary file contains executable code for a custom virtual machine. Regardless, it is safe to say that the .exe program takes in code from injector.bin to encrypt MasterKey.png.

Moving on, analyzing the .exe file in strings, the user will see this:

```

L$ strings BlackShell.exe
!This program cannot be run in DOS mode.
UPX0
UPX1
UPX2
4.24
UPX!
1Lf:
8MZu
HcP<H
xt:D
ATUx3
WVSU 1
[^_]A\
N:<\hj
F8B6
800>; L8S
{'c
`v \pzT
q8!w
0{XK
J          ]9
2Ar(y
fI6I
m4Rj
rr?B
6vqL9-
yw.b
3"!W
```

In the first few strings of the binary, the user can identify that the .exe has been UPX-packed. UPX is a popular executable packer which compresses and shrinks the original bytes of the file.

With this, let's unpack the .exe:

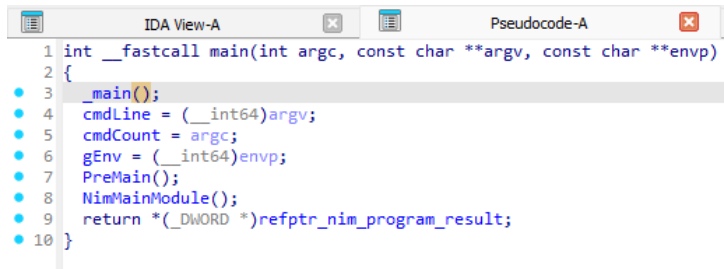
```

L$ upx -d BlackShell.exe
                Ultimate Packer for eXecutables
                Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser    May 9th 2024

  File size      Ratio      Format      Name
  -----
  404000 ←    223264    55.26%    win64/pe    BlackShell.exe

Unpacked 1 file.
```

Decompiling this with a decompiler (I'll be using IDA here), the user will be greeted with the following:



```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     __main();
4     cmdLine = (__int64)argv;
5     cmdCount = argc;
6     gEnv = (__int64)envp;
7     PreMain();
8     NimMainModule();
9     return *(_DWORD *)refptr_nim_program_result;
10 }
```

From here, we can see the wrapper for the 'main' of Nim programs. With some simple google searching the user can quickly find that the real entry point is in `NimMainModule()` :



```
8
9 v4 = __mm_loadu_si128((__m128i *)&TM_1HbMekVh86zcR9bQ2e2UEqg_3);
10 newWideCString__stdZwiderstrs_u325(&v5, &v4);
11 v0 = __mm_loadu_si128(&v5);
12 address = (__BYTE *)__emutls_get_address(refptr__emutls_v_nimInErrorMode__system_u4398);
13 if ( !*address )
14     goto LABEL_8;
15 DS_STREAM_RENAME__66lack83hell_u23 = (__int128)v0;
16 executeProtocol__66lack83hell_u5025();
17 if ( !*address )
18 {
19     randomize__pureZrandom_u377();
20     if ( !*address )
21     {
22         v2 = __acrt_iob_func(1u);
23         hideCursor__pureZterminal_u312(v2);
24         if ( !*address )
25         {
26             BOHBanner__66lack83hell_u196_part_0();
27             if ( !*address )
28             {
29                 nossleep(3000);
30                 if ( !*address )
31                 {
32                     v5 = __mm_loadu_si128((__m128i *)&DS_STREAM_RENAME__66lack83hell_u23);
33 LABEL_8:
34                     eqdestroy__stdZwiderstrs_u7(&v5);
35                 }
36             }
37         }
38     }
39 }
```

Immediately, the user may find that the decompiled code has some **extra** texts compared to standard decompilation. As the program was not originally written in C/C++, this is the decompiler's (IDA), best effort in converting the Nim code into C pseudocode. This adds some difficulty into static decompilation, but it's still readable regardless.

Here we can see 4 functions in our decompiled code: `executeProtocol`, `hideCursor` and `glitchBanner`, where aside from `executeProtocol`, the rest looks like functions that handled .exe's decor. Let's jump into the function.

```

106 v72 = (unsigned __int128)_mm_loadu_si128((const __m128i *)&TM__HbMekVh86zcR9bQ2e2UEqg_7);
107 v7 = winstrConverterStringToLPWSTR__pkgZwinimZwinstr_u1370(&v72);
108 if ( !*address )
109 {
110     ((void (__fastcall *)(_QWORD, __int64, __int64, _QWORD))*refptr_D1_2667585454_)(0, v6, v7, 0);
111     exit(1);
112 }
113 }
114 goto LABEL_10;
115 }
116 v8 = (const __m128i *)&unk_14002BCC8;
117 do
118 {
119     v72 = (unsigned __int128)_mm_loadu_si128(v8);
120     v9 = nosdirExists(&v72);
121     if ( !*address )
122         goto LABEL_10;
123     if ( v9 )
124     {
125         memset(v80, 0, 0x20Au);
126         if ( (__fastcall int (__fastcall *)(_QWORD, _WORD *, __int64))*refptr_D1_2634025382_)(0, v80, 260) )
127         {
128             v13 = ((__int64 (__fastcall *)(_QWORD *, __int64, _QWORD, _QWORD, int, int, _QWORD))*refptr_D1_2634024909_)(
129                 v80,
130                 0x10000,
131                 0,
132                 0,
133                 3,
134                 128,
135                 0);
136             v14 = v13;
137             if ( !*address )
138                 goto LABEL_10;
139         }
140     }
141 } while (v9);
142 }
143 }
144 }
145 }
146 }
147 }
148 }
149 }
150 }
151 }
152 }
153 }
154 }
155 }
156 }
157 }
158 }
159 }
160 }
161 }
162 }
163 }
164 }
165 }
166 }
167 }
168 }
169 }
170 }
171 }
172 }
173 }
174 }
175 }
176 }
177 }
178 }
179 }
180 }
181 }
182 }
183 }
184 }
185 }
186 }
187 }
188 }
189 }
190 }
191 }
192 }
193 }
194 }
195 }
196 }
197 }
198 }
199 }
200 }
201 }
202 }
203 }
204 }
205 }
206 }
207 }
208 }
209 }
210 }
211 }
212 }
213 }
214 }
215 }
216 }
217 }
218 }
219 }
220 }
221 }
222 }
223 }
224 }
225 }
226 }
227 }
228 }
229 }
230 }
231 }
232 }
233 }
234 }
235 }
236 }
237 }
238 }
239 }
240 }
241 }
242 }
243 }
244 }
245 }
246 }
247 }
248 }
249 }
250 }
251 }
252 }
253 }
254 }
255 }
256 }
257 }
258 }
259 }
260 }
261 }
262 }
263 }
264 }
265 }
266 }
267 }
268 }
269 }
270 }
271 }
272 }
273 }
274 }
275 }
276 }
277 }
278 }
279 }
280 }
281 }
282 }
283 }
284 }
285 }
286 }
287 }
288 }
289 }
290 }
291 }
292 }
293 }
294 }
295 }
296 }
297 }
298 }
299 }
300 }
301 }
302 }
303 }
304 }
305 }
306 }
307 }
308 }
309 }
310 }
311 }
312 }
313 }
314 }
315 }
316 }
317 }
318 }
319 }
320 }
321 }
322 }
323 }
324 }
325 }
326 }
327 }
328 }
329 }
330 }
331 }
332 }
333 }
334 }
335 }
336 }
337 }
338 }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }

```

Going deeper, we can find this part of the code:

```

378 }
379 LABEL_50:
380 v66 = v18;
381 v63 = v24;
382 v57 = v23;
383 *(_QWORD *)&v77[0] = 0x200000001LL;
384 *(_QWORD *)&v77[0] + 1 = 0x400000003LL;
385 eqwasMoved__pureZtimes_u2665(&v73);
386 v78.m128i_i64[0] = v57;
387 v78.m128i_i64[1] = v63;
388 virtualSetup__66lack83hell_u3166(v77);
389 v18 = v66;
390 if ( !*address )
391     goto LABEL_93;
392 deriveNeuralKeyPair__66lack83hell_u3189(v77, &v75);
393 v18 = v66;
394 if ( !*address )
395     goto LABEL_93;

```

Here we can see two functions `virtualSetup` and `deriveNeuralKeyPair`.

```

486     goto LABEL_85;
487     v70 = v72;
488     v71 = __mm_loadu_si128(v79);
489     nosjoinPath(&v72, &v71, &v70);
490     if ( *address )
491         goto LABEL_85;
492     eqsink__system_u2665(&v74, &v72);
493     v72 = (unsigned __int128)__mm_loadu_si128(&v74);
494     v38 = isPngFile__66lack83hell_u201(&v72);
495     if ( *address
496         || v38
497         && (v39 = __mm_loadu_si128(&v74),
498             v71 = v46,
499             v72 = (unsigned __int128)v39,
500             v70 = (unsigned __int128)v47,
501             executeAlgorithmicEnigmaSystem__66lack83hell_u3698(&v72, &v71, &v70),
502             *address) )
503     {
504 LABEL_85:
505         v31 = v33;
506         v1 = v32;
507         goto LABEL_86;
508     }

```

If we head deeper into the `ingniteProtocol`` function, we can find another function called `executeAlgorithmicEnigmaSystem``, a hint to the AES encryption.

If we head back to where the `virtualSetup`` function resides and clicking it, we go into a new page of code:

```

1 void __fastcall virtualSetup__66lack83hell_u3166(_QWORD *a1)
2 {
3     __int64 v1; // rdx
4     __int64 i; // rcx
5     __int64 v4; // rdx
6
7     v1 = a1[4];
8     if ( v1 <= 0 || (v1 & 0x1F) != 0 )
9     {
10         a1[2] = 0;
11         for ( i = 0; v1 > i; i = a1[2] )
12         {
13             if ( i < 0 )
14             {
15                 raiseIndexError2(i, v1 - 1);
16                 return;
17             }
18             v4 = *(unsigned __int8 *)(a1[5] + i + 8);
19             a1[2] = i + 1;
20             executeShell__66lack83hell_u364(a1, v4);
21             if ( *(_BYTE *)__emutls_get_address(refptr__emutls_v_nimInErrorMode__system_u4398) )
22                 return;
23             v1 = a1[4];
24         }
25     }
26     else
27     {
28         initMaskedShell__66lack83hell_u3072_part_0();
29     }
30 }

```

We're able to see two functions which are `executeShell`` and `initMaskedShell``.

Inside `executeShell`, we find a very messy VM bytecode map:

```
42
43 if ( a2 == 0xB2 )
44 {
45     v33 = __ROL4__(a1->m128i_i32[1], 5);
46     if ( !*(__BYTE *)_emutls_get_address(refptr__emutls_v_nimInErrorMode__system_u4398) )
47     {
48         a1->m128i_i32[0] = v33 + (a1->m128i_i32[0] ^ 0xC3EF5D7A);
49         a1->m128i_i32[2] = (a1->m128i_i32[2] + a1->m128i_i32[3]) ^ 0xA5B4C3D2;
50     }
51 }
52 else if ( a2 <= 0xB2u )
53 {
54     if ( a2 == 0x91 )
55     {
56         v25 = a1[2].m128i_i64[0];
57         if ( v25 > 0 )
58         {
59             v26 = a1->m128i_i32[0] % (unsigned int)v25;
60             if ( v25 <= (unsigned int)v26 )
61                 raiseIndexError2(v26, v25 - 1);
62             else
63                 *(__BYTE *) (a1[2].m128i_i64[1] + v26 + 8) ^= a1->m128i_u32[1];
64         }
65     }
66     else if ( a2 > 0x91u )
67     {
68         if ( a2 == 0xA7 )
69             a1->m128i_i32[2] = a1->m128i_i32[0] - 1640531535 * a1->m128i_i32[2];
70     }
71     else if ( a2 == 108 )
72     {
73         v14 = mm_loadu_si128(a1);
74     }
75 }
```

Going through it, we can piece it into a 9-byte instruction map of the following:

`, ` , `

- **0xCA** → Adds AX and BX, subtracts DX from CX, XORs the results, and stores back into AX (also updates flags).
- **0xFE** → Rotates AX left by the number of bits in BX (lowest 5 bits), and sets the carry flag depending on the result.
- **0xA7** → Multiplies CX by a golden-ratio constant (0x9E3779B1) and adds AX to it (TEA-style step).
- **0xD3** → Concatenates all four registers, runs SHA-1, and stores the first 4 bytes of the digest into DX.
- **0x91** → Picks a memory index from AX modulo memory size, and XORs that byte with BX's low byte.
- **0x6C** → Computes a mix of all registers and conditionally advances the instruction pointer by 1 or 2.
- **0xE4** → If the trap flag is set, XORs BX with the magic constant 0xBADC0FFE.
- **0xB2** → Scrambles registers: $AX = (AX \oplus 0xC3EF5D7A) + \text{ROL5}(BX)$, $CX = (CX + DX) \oplus 0xA5B4C3D2$.
- **0x7F** → Mixes AX and BX, then reduces BX modulo 65521 (largest 16-bit prime).
- **0xFF** → Essentially HALT instruction, marking the end of execution.

`, ` , `

After all that, at the very end of the VM bytecode function, we can see a `finish` function, let's jump into it:

```
227 LABEL_52:
228     if ( !v34 )
229     {
230 LABEL_53:
231     finish__66lack83hell_u2774(v40, v41);
232     if ( !*v28 )
233     {
234         a1->m128i_i32[3] = v40[0];
235     }
236 }

16 v14 = 0;
17 v13 = 0;
18 v5 = v2 & 0x3F;
19 if ( (v2 & 0x3F) != 0 )
20 {
21     *((_BYTE *)a2 + v5 + 28) = 0x80;
22     if ( v5 == 63 )
23     {
24         sha1Transform__66lack83hell_u501(a2, (char *)a2 + 28, 64, 0, 0, 0, 0);
25         address = (char *)_emutls_get_address(refptr__emutls_v_nimInErrorMode__system_u4398);
26         v7 = *address;
27         goto LABEL_4;
28     }
29 }
30 else
31 {
32     *((_BYTE *)a2 + 28) = 0x80;
33 }
34 address = (char *)_emutls_get_address(refptr__emutls_v_nimInErrorMode__system_u4398);
35 v7 = *address;
36 LABEL_4:
37 if ( v7 )
38     goto LABEL_9;
39 while ( 1 )
40 {
41     v8 = *a2;
42     v9 = *a2 & 0x3F;
43     if ( v9 == 56 )
44         break;
45     while ( 1 )
46     {
47         *a2 = v8 + 1;
48         if ( *a2 == 0 )
49             *a2 = 0xFF;
50         a2++;
51     }
52     v8 = *a2;
53     v9 = *a2 & 0x3F;
54     if ( v9 == 56 )
55         break;
56     while ( 1 )
57     {
58         *a2 = v8 + 1;
59         if ( *a2 == 0 )
60             *a2 = 0xFF;
61         a2++;
62     }
63     v8 = *a2;
64     v9 = *a2 & 0x3F;
65     if ( v9 == 56 )
66         break;
67     while ( 1 )
68     {
69         *a2 = v8 + 1;
70         if ( *a2 == 0 )
71             *a2 = 0xFF;
72         a2++;
73     }
74     v8 = *a2;
75     v9 = *a2 & 0x3F;
76     if ( v9 == 56 )
77         break;
78     while ( 1 )
79     {
80         *a2 = v8 + 1;
81         if ( *a2 == 0 )
82             *a2 = 0xFF;
83         a2++;
84     }
85     v8 = *a2;
86     v9 = *a2 & 0x3F;
87     if ( v9 == 56 )
88         break;
89     while ( 1 )
90     {
91         *a2 = v8 + 1;
92         if ( *a2 == 0 )
93             *a2 = 0xFF;
94         a2++;
95     }
96     v8 = *a2;
97     v9 = *a2 & 0x3F;
98     if ( v9 == 56 )
99         break;
100    while ( 1 )
101    {
102        *a2 = v8 + 1;
103        if ( *a2 == 0 )
104            *a2 = 0xFF;
105        a2++;
106    }
107    v8 = *a2;
108    v9 = *a2 & 0x3F;
109    if ( v9 == 56 )
110        break;
111    while ( 1 )
112    {
113        *a2 = v8 + 1;
114        if ( *a2 == 0 )
115            *a2 = 0xFF;
116        a2++;
117    }
118    v8 = *a2;
119    v9 = *a2 & 0x3F;
120    if ( v9 == 56 )
121        break;
122    while ( 1 )
123    {
124        *a2 = v8 + 1;
125        if ( *a2 == 0 )
126            *a2 = 0xFF;
127        a2++;
128    }
129    v8 = *a2;
130    v9 = *a2 & 0x3F;
131    if ( v9 == 56 )
132        break;
133    while ( 1 )
134    {
135        *a2 = v8 + 1;
136        if ( *a2 == 0 )
137            *a2 = 0xFF;
138        a2++;
139    }
140    v8 = *a2;
141    v9 = *a2 & 0x3F;
142    if ( v9 == 56 )
143        break;
144    while ( 1 )
145    {
146        *a2 = v8 + 1;
147        if ( *a2 == 0 )
148            *a2 = 0xFF;
149        a2++;
150    }
151    v8 = *a2;
152    v9 = *a2 & 0x3F;
153    if ( v9 == 56 )
154        break;
155    while ( 1 )
156    {
157        *a2 = v8 + 1;
158        if ( *a2 == 0 )
159            *a2 = 0xFF;
160        a2++;
161    }
162    v8 = *a2;
163    v9 = *a2 & 0x3F;
164    if ( v9 == 56 )
165        break;
166    while ( 1 )
167    {
168        *a2 = v8 + 1;
169        if ( *a2 == 0 )
170            *a2 = 0xFF;
171        a2++;
172    }
173    v8 = *a2;
174    v9 = *a2 & 0x3F;
175    if ( v9 == 56 )
176        break;
177    while ( 1 )
178    {
179        *a2 = v8 + 1;
180        if ( *a2 == 0 )
181            *a2 = 0xFF;
182        a2++;
183    }
184    v8 = *a2;
185    v9 = *a2 & 0x3F;
186    if ( v9 == 56 )
187        break;
188    while ( 1 )
189    {
190        *a2 = v8 + 1;
191        if ( *a2 == 0 )
192            *a2 = 0xFF;
193        a2++;
194    }
195    v8 = *a2;
196    v9 = *a2 & 0x3F;
197    if ( v9 == 56 )
198        break;
199    while ( 1 )
200    {
201        *a2 = v8 + 1;
202        if ( *a2 == 0 )
203            *a2 = 0xFF;
204        a2++;
205    }
206    v8 = *a2;
207    v9 = *a2 & 0x3F;
208    if ( v9 == 56 )
209        break;
210    while ( 1 )
211    {
212        *a2 = v8 + 1;
213        if ( *a2 == 0 )
214            *a2 = 0xFF;
215        a2++;
216    }
217    v8 = *a2;
218    v9 = *a2 & 0x3F;
219    if ( v9 == 56 )
220        break;
221    while ( 1 )
222    {
223        *a2 = v8 + 1;
224        if ( *a2 == 0 )
225            *a2 = 0xFF;
226        a2++;
227    }
228    v8 = *a2;
229    v9 = *a2 & 0x3F;
230    if ( v9 == 56 )
231        break;
232    while ( 1 )
233    {
234        *a2 = v8 + 1;
235        if ( *a2 == 0 )
236            *a2 = 0xFF;
237        a2++;
238    }
239    v8 = *a2;
240    v9 = *a2 & 0x3F;
241    if ( v9 == 56 )
242        break;
243    while ( 1 )
244    {
245        *a2 = v8 + 1;
246        if ( *a2 == 0 )
247            *a2 = 0xFF;
248        a2++;
249    }
250    v8 = *a2;
251    v9 = *a2 & 0x3F;
252    if ( v9 == 56 )
253        break;
254    while ( 1 )
255    {
256        *a2 = v8 + 1;
257        if ( *a2 == 0 )
258            *a2 = 0xFF;
259        a2++;
260    }
261    v8 = *a2;
262    v9 = *a2 & 0x3F;
263    if ( v9 == 56 )
264        break;
265    while ( 1 )
266    {
267        *a2 = v8 + 1;
268        if ( *a2 == 0 )
269            *a2 = 0xFF;
270        a2++;
271    }
272    v8 = *a2;
273    v9 = *a2 & 0x3F;
274    if ( v9 == 56 )
275        break;
276    while ( 1 )
277    {
278        *a2 = v8 + 1;
279        if ( *a2 == 0 )
280            *a2 = 0xFF;
281        a2++;
282    }
283    v8 = *a2;
284    v9 = *a2 & 0x3F;
285    if ( v9 == 56 )
286        break;
287    while ( 1 )
288    {
289        *a2 = v8 + 1;
290        if ( *a2 == 0 )
291            *a2 = 0xFF;
292        a2++;
293    }
294    v8 = *a2;
295    v9 = *a2 & 0x3F;
296    if ( v9 == 56 )
297        break;
298    while ( 1 )
299    {
300        *a2 = v8 + 1;
301        if ( *a2 == 0 )
302            *a2 = 0xFF;
303        a2++;
304    }
305    v8 = *a2;
306    v9 = *a2 & 0x3F;
307    if ( v9 == 56 )
308        break;
309    while ( 1 )
310    {
311        *a2 = v8 + 1;
312        if ( *a2 == 0 )
313            *a2 = 0xFF;
314        a2++;
315    }
316    v8 = *a2;
317    v9 = *a2 & 0x3F;
318    if ( v9 == 56 )
319        break;
320    while ( 1 )
321    {
322        *a2 = v8 + 1;
323        if ( *a2 == 0 )
324            *a2 = 0xFF;
325        a2++;
326    }
327    v8 = *a2;
328    v9 = *a2 & 0x3F;
329    if ( v9 == 56 )
330        break;
331    while ( 1 )
332    {
333        *a2 = v8 + 1;
334        if ( *a2 == 0 )
335            *a2 = 0xFF;
336        a2++;
337    }
338    v8 = *a2;
339    v9 = *a2 & 0x3F;
340    if ( v9 == 56 )
341        break;
342    while ( 1 )
343    {
344        *a2 = v8 + 1;
345        if ( *a2 == 0 )
346            *a2 = 0xFF;
347        a2++;
348    }
349    v8 = *a2;
350    v9 = *a2 & 0x3F;
351    if ( v9 == 56 )
352        break;
353    while ( 1 )
354    {
355        *a2 = v8 + 1;
356        if ( *a2 == 0 )
357            *a2 = 0xFF;
358        a2++;
359    }
360    v8 = *a2;
361    v9 = *a2 & 0x3F;
362    if ( v9 == 56 )
363        break;
364    while ( 1 )
365    {
366        *a2 = v8 + 1;
367        if ( *a2 == 0 )
368            *a2 = 0xFF;
369        a2++;
370    }
371    v8 = *a2;
372    v9 = *a2 & 0x3F;
373    if ( v9 == 56 )
374        break;
375    while ( 1 )
376    {
377        *a2 = v8 + 1;
378        if ( *a2 == 0 )
379            *a2 = 0xFF;
380        a2++;
381    }
382    v8 = *a2;
383    v9 = *a2 & 0x3F;
384    if ( v9 == 56 )
385        break;
386    while ( 1 )
387    {
388        *a2 = v8 + 1;
389        if ( *a2 == 0 )
390            *a2 = 0xFF;
391        a2++;
392    }
393    v8 = *a2;
394    v9 = *a2 & 0x3F;
395    if ( v9 == 56 )
396        break;
397    while ( 1 )
398    {
399        *a2 = v8 + 1;
400        if ( *a2 == 0 )
401            *a2 = 0xFF;
402        a2++;
403    }
404    v8 = *a2;
405    v9 = *a2 & 0x3F;
406    if ( v9 == 56 )
407        break;
408    while ( 1 )
409    {
410        *a2 = v8 + 1;
411        if ( *a2 == 0 )
412            *a2 = 0xFF;
413        a2++;
414    }
415    v8 = *a2;
416    v9 = *a2 & 0x3F;
417    if ( v9 == 56 )
418        break;
419    while ( 1 )
420    {
421        *a2 = v8 + 1;
422        if ( *a2 == 0 )
423            *a2 = 0xFF;
424        a2++;
425    }
426    v8 = *a2;
427    v9 = *a2 & 0x3F;
428    if ( v9 == 56 )
429        break;
430    while ( 1 )
431    {
432        *a2 = v8 + 1;
433        if ( *a2 == 0 )
434            *a2 = 0xFF;
435        a2++;
436    }
437    v8 = *a2;
438    v9 = *a2 & 0x3F;
439    if ( v9 == 56 )
440        break;
441    while ( 1 )
442    {
443        *a2 = v8 + 1;
444        if ( *a2 == 0 )
445            *a2 = 0xFF;
446        a2++;
447    }
448    v8 = *a2;
449    v9 = *a2 & 0x3F;
450    if ( v9 == 56 )
451        break;
452    while ( 1 )
453    {
454        *a2 = v8 + 1;
455        if ( *a2 == 0 )
456            *a2 = 0xFF;
457        a2++;
458    }
459    v8 = *a2;
460    v9 = *a2 & 0x3F;
461    if ( v9 == 56 )
462        break;
463    while ( 1 )
464    {
465        *a2 = v8 + 1;
466        if ( *a2 == 0 )
467            *a2 = 0xFF;
468        a2++;
469    }
470    v8 = *a2;
471    v9 = *a2 & 0x3F;
472    if ( v9 == 56 )
473        break;
474    while ( 1 )
475    {
476        *a2 = v8 + 1;
477        if ( *a2 == 0 )
478            *a2 = 0xFF;
479        a2++;
480    }
481    v8 = *a2;
482    v9 = *a2 & 0x3F;
483    if ( v9 == 56 )
484        break;
485    while ( 1 )
486    {
487        *a2 = v8 + 1;
488        if ( *a2 == 0 )
489            *a2 = 0xFF;
490        a2++;
491    }
492    v8 = *a2;
493    v9 = *a2 & 0x3F;
494    if ( v9 == 56 )
495        break;
496    while ( 1 )
497    {
498        *a2 = v8 + 1;
499        if ( *a2 == 0 )
500            *a2 = 0xFF;
501        a2++;
502    }
503    v8 = *a2;
504    v9 = *a2 & 0x3F;
505    if ( v9 == 56 )
506        break;
507    while ( 1 )
508    {
509        *a2 = v8 + 1;
510        if ( *a2 == 0 )
511            *a2 = 0xFF;
512        a2++;
513    }
514    v8 = *a2;
515    v9 = *a2 & 0x3F;
516    if ( v9 == 56 )
517        break;
518    while ( 1 )
519    {
520        *a2 = v8 + 1;
521        if ( *a2 == 0 )
522            *a2 = 0xFF;
523        a2++;
524    }
525    v8 = *a2;
526    v9 = *a2 & 0x3F;
527    if ( v9 == 56 )
528        break;
529    while ( 1 )
530    {
531        *a2 = v8 + 1;
532        if ( *a2 == 0 )
533            *a2 = 0xFF;
534        a2++;
535    }
536    v8 = *a2;
537    v9 = *a2 & 0x3F;
538    if ( v9 == 56 )
539        break;
540    while ( 1 )
541    {
542        *a2 = v8 + 1;
543        if ( *a2 == 0 )
544            *a2 = 0xFF;
545        a2++;
546    }
547    v8 = *a2;
548    v9 = *a2 & 0x3F;
549    if ( v9 == 56 )
550        break;
551    while ( 1 )
552    {
553        *a2 = v8 + 1;
554        if ( *a2 == 0 )
555            *a2 = 0xFF;
556        a2++;
557    }
558    v8 = *a2;
559    v9 = *a2 & 0x3F;
560    if ( v9 == 56 )
561        break;
562    while ( 1 )
563    {
564        *a2 = v8 + 1;
565        if ( *a2 == 0 )
566            *a2 = 0xFF;
567        a2++;
568    }
569    v8 = *a2;
570    v9 = *a2 & 0x3F;
571    if ( v9 == 56 )
572        break;
573    while ( 1 )
574    {
575        *a2 = v8 + 1;
576        if ( *a2 == 0 )
577            *a2 = 0xFF;
578        a2++;
579    }
580    v8 = *a2;
581    v9 = *a2 & 0x3F;
582    if ( v9 == 56 )
583        break;
584    while ( 1 )
585    {
586        *a2 = v8 + 1;
587        if ( *a2 == 0 )
588            *a2 = 0xFF;
589        a2++;
590    }
591    v8 = *a2;
592    v9 = *a2 & 0x3F;
593    if ( v9 == 56 )
594        break;
595    while ( 1 )
596    {
597        *a2 = v8 + 1;
598        if ( *a2 == 0 )
599            *a2 = 0xFF;
600        a2++;
601    }
602    v8 = *a2;
603    v9 = *a2 & 0x3F;
604    if ( v9 == 56 )
605        break;
606    while ( 1 )
607    {
608        *a2 = v8 + 1;
609        if ( *a2 == 0 )
610            *a2 = 0xFF;
611        a2++;
612    }
613    v8 = *a2;
614    v9 = *a2 & 0x3F;
615    if ( v9 == 56 )
616        break;
617    while ( 1 )
618    {
619        *a2 = v8 + 1;
620        if ( *a2 == 0 )
621            *a2 = 0xFF;
622        a2++;
623    }
624    v8 = *a2;
625    v9 = *a2 & 0x3F;
626    if ( v9 == 56 )
627        break;
628    while ( 1 )
629    {
630        *a2 = v8 + 1;
631        if ( *a2 == 0 )
632            *a2 = 0xFF;
633        a2++;
634    }
635    v8 = *a2;
636    v9 = *a2 & 0x3F;
637    if ( v9 == 56 )
638        break;
639    while ( 1 )
640    {
641        *a2 = v8 + 1;
642        if ( *a2 == 0 )
643            *a2 = 0xFF;
644        a2++;
645    }
646    v8 = *a2;
647    v9 = *a2 & 0x3F;
648    if ( v9 == 56 )
649        break;
650    while ( 1 )
651    {
652        *a2 = v8 + 1;
653        if ( *a2 == 0 )
654            *a2 = 0xFF;
655        a2++;
656    }
657    v8 = *a2;
658    v9 = *a2 & 0x3F;
659    if ( v9 == 56 )
660        break;
661    while ( 1 )
662    {
663        *a2 = v8 + 1;
664        if ( *a2 == 0 )
665            *a2 = 0xFF;
666        a2++;
667    }
668    v8 = *a2;
669    v9 = *a2 & 0x3F;
670    if ( v9 == 56 )
671        break;
672    while ( 1 )
673    {
674        *a2 = v8 + 1;
675        if ( *a2 == 0 )
676            *a2 = 0xFF;
677        a2++;
678    }
679    v8 = *a2;
680    v9 = *a2 & 0x3F;
681    if ( v9 == 56 )
682        break;
683    while ( 1 )
684    {
685        *a2 = v8 + 1;
686        if ( *a2 == 0 )
687            *a2 = 0xFF;
688        a2++;
689    }
690    v8 = *a2;
691    v9 = *a2 & 0x3F;
692    if ( v9 == 56 )
693        break;
694    while ( 1 )
695    {
696        *a2 = v8 + 1;
697        if ( *a2 == 0 )
698            *a2 = 0xFF;
699        a2++;
700    }
701    v8 = *a2;
702    v9 = *a2 & 0x3F;
703    if ( v9 == 56 )
704        break;
705    while ( 1 )
706    {
707        *a2 = v8 + 1;
708        if ( *a2 == 0 )
709            *a2 = 0xFF;
710        a2++;
711    }
712    v8 = *a2;
713    v9 = *a2 & 0x3F;
714    if ( v9 == 56 )
715        break;
716    while ( 1 )
717    {
718        *a2 = v8 + 1;
719        if ( *a2 == 0 )
720            *a2 = 0xFF;
721        a2++;
722    }
723    v8 = *a2;
724    v9 = *a2 & 0x3F;
725    if ( v9 == 56 )
726        break;
727    while ( 1 )
728    {
729        *a2 = v8 + 1;
730        if ( *a2 == 0 )
731            *a2 = 0xFF;
732        a2++;
733    }
734    v8 = *a2;
735    v9 = *a2 & 0x3F;
736    if ( v9 == 56 )
737        break;
738    while ( 1 )
739    {
740        *a2 = v8 + 1;
741        if ( *a2 == 0 )
742            *a2 = 0xFF;
743        a2++;
744    }
745    v8 = *a2;
746    v9 = *a2 & 0x3F;
747    if ( v9 == 56 )
748        break;
749    while ( 1 )
750    {
751        *a2 = v8 + 1;
752        if ( *a2 == 0 )
753            *a2 = 0xFF;
754        a2++;
755    }
756    v8 = *a2;
757    v9 = *a2 & 0x3F;
758    if ( v9 == 56 )
759        break;
760    while ( 1 )
761    {
762        *a2 = v8 + 1;
763        if ( *a2 == 0 )
764            *a2 = 0xFF;
765        a2++;
766    }
767    v8 = *a2;
768    v9 = *a2 & 0x3F;
769    if ( v9 == 56 )
770        break;
771    while ( 1 )
772    {
773        *a2 = v8 + 1;
774        if ( *a2 == 0 )
775            *a2 = 0xFF;
776        a2++;
777    }
778    v8 = *a2;
779    v9 = *a2 & 0x3F;
780    if ( v9 == 56 )
781        break;
782    while ( 1 )
783    {
784        *a2 = v8 + 1;
785        if ( *a2 == 0 )
786            *a2 = 0xFF;
787        a2++;
788    }
789    v8 = *a2;
790    v9 = *a2 & 0x3F;
791    if ( v9 == 56 )
792        break;
793    while ( 1 )
794    {
795        *a2 = v8 + 1;
796        if ( *a2 == 0 )
797            *a2 = 0xFF;
798        a2++;
799    }
800    v8 = *a2;
801    v9 = *a2 & 0x3F;
802    if ( v9 == 56 )
803        break;
804    while ( 1 )
805    {
806        *a2 = v8 + 1;
807        if ( *a2 == 0 )
808            *a2 = 0xFF;
809        a2++;
810    }
811    v8 = *a2;
812    v9 = *a2 & 0x3F;
813    if ( v9 == 56 )
814        break;
815    while ( 1 )
816    {
817        *a2 = v8 + 1;
818        if ( *a2 == 0 )
819            *a2 = 0xFF;
820        a2++;
821    }
822    v8 = *a2;
823    v9 = *a2 & 0x3F;
824    if ( v9 == 56 )
825        break;
826    while ( 1 )
827    {
828        *a2 = v8 + 1;
829        if ( *a2 == 0 )
830            *a2 = 0xFF;
831        a2++;
832    }
833    v8 = *a2;
834    v9 = *a2 & 0x3F;
835    if ( v9 == 56 )
836        break;
837    while ( 1 )
838    {
839        *a2 = v8 + 1;
840        if ( *a2 == 0 )
841            *a2 = 0xFF;
842        a2++;
843    }
844    v8 = *a2;
845    v9 = *a2 & 0x3F;
846    if ( v9 == 56 )
847        break;
848    while ( 1 )
849    {
850        *a2 = v8 + 1;
851        if ( *a2 == 0 )
852            *a2 = 0xFF;
853        a2++;
854    }
855    v8 = *a2;
856    v9 = *a2 & 0x3F;
857    if ( v9 == 56 )
858        break;
859    while ( 1 )
860    {
861        *a2 = v8 + 1;
862        if ( *a2 == 0 )
863            *a2 = 0xFF;
864        a2++;
865    }
866    v8 = *a2;
867    v9 = *a2 & 0x3F;
868    if ( v9 == 56 )
869        break;
870    while ( 1 )
871    {
872        *a2 = v8 + 1;
873        if ( *a2 == 0 )
874            *a2 = 0xFF;
875        a2++;
876    }
877    v8 = *a2;
878    v9 = *a2 & 0x3F;
879    if ( v9 == 56 )
880        break;
881    while ( 1 )
882    {
883        *a2 = v8 + 1;
884        if ( *a2 == 0 )
885            *a2 = 0xFF;
886        a2++;
887    }
888    v8 = *a2;
889    v9 = *a2 & 0x3F;
890    if ( v9 == 56 )
891        break;
892    while ( 1 )
893    {
894        *a2 = v8 + 1;
895        if ( *a2 == 0 )
896            *a2 = 0xFF;
897        a2++;
898    }
899    v8 = *a2;
900    v9 = *a2 & 0x3F;
901    if ( v9 == 56 )
902        break;
903    while ( 1 )
904    {
905        *a2 = v8 + 1;
906        if ( *a2 == 0 )
907            *a2 = 0xFF;
908        a2++;
909    }
910    v8 = *a2;
911    v9 = *a2 & 0x3F;
912    if ( v9 == 56 )
913        break;
914    while ( 1 )
915    {
916        *a2 = v8 + 1;
917        if ( *a2 == 0 )
918            *a2 = 0xFF;
919        a2++;
920    }
921    v8 = *a2;
922    v9 = *a2 & 0x3F;
923    if ( v9 == 56 )
924        break;
925    while ( 1 )
926    {
927        *a2 = v8 + 1;
928        if ( *a2 == 0 )
929            *a2 = 0xFF;
930        a2++;
931    }
932    v8 = *a2;
933    v9 = *a2 & 0x3F;
934    if ( v9 == 56 )
935        break;
936    while ( 1 )
937    {
938        *a2 = v8 + 1;
939        if ( *a2 == 0 )
940            *a2 = 0xFF;
941        a2++;
942    }
943    v8 = *a2;
944    v9 = *a2 & 0x3F;
945    if ( v9 == 56 )
946        break;
947    while ( 1 )
948    {
949        *a2 = v8 + 1;
950        if ( *a2 == 0 )
951            *a2 = 0xFF;
952        a2++;
953    }
954    v8 = *a2;
955    v9 = *a2 & 0x3F;
956    if ( v9 == 56 )
957        break;
958    while ( 1 )
959    {
960        *a2 = v8 + 1;
961        if ( *a2 == 0 )
962            *a2 = 0xFF;
963        a2++;
964    }
965    v8 = *a2;
966    v9 = *a2 & 0x3F;
967    if ( v9 == 56 )
968        break;
969    while ( 1 )
970    {
971        *a2 = v8 + 1;
972        if ( *a2 == 0 )
973            *a2 = 0xFF;
974        a2++;
975    }
976    v8 = *a2;
977    v9 = *a2 & 0x3F;
978    if ( v9 == 56 )
979        break;
980    while ( 1 )
981    {
982        *a2 = v8 + 1;
983        if ( *a2 == 0 )
984            *a2 = 0xFF;
985        a2++;
986    }
987    v8 = *a2;
988    v9 = *a2 & 0x3F;
989    if ( v9 == 56 )
990        break;
991    while ( 1 )
992    {
993        *a2 = v8 + 1;
994        if ( *a2 == 0 )
995            *a2 = 0xFF;
996        a2++;
997    }
998    v8 = *a2;
999    v9 = *a2 & 0x3F;
1000    if ( v9 == 56 )
1001        break;
1002    while ( 1 )
1003    {
1004        *a2 = v8 + 1;
1005        if ( *a2 == 0 )
1006            *a2 = 0xFF;
1007        a2++;
1008    }
1009    v8 = *a2;
1010    v9 = *a2 & 0x3F;
1011    if ( v9 == 56 )
1012        break;
1013    while ( 1 )
1014    {
1015        *a2 = v8 + 1;
1016        if ( *a2 == 0 )
1017            *a2 = 0xFF;
1018        a2++;
1019    }
1020    v8 = *a2;
1021    v9 = *a2 & 0x3F;
1022    if ( v9 == 56 )
1023        break;
1024    while ( 1 )
1025    {
1026        *a2 = v8 + 1;
1027        if ( *a2 == 0 )
1028            *a2 = 0xFF;
1029        a2++;
1030    }
1031    v8 = *a2;
1032    v9 = *a2 & 0x3F;
1033    if ( v9 == 56 )
1034        break;
1035    while ( 1 )
1036    {
1037        *a2 = v8 + 1;
1038        if ( *a2 == 0 )
1039            *a2 = 0xFF;
1040        a2++;
1041    }
1042    v8 = *a2;
1043    v9 = *a2 & 0x3F;
1044    if ( v9 == 56 )
1045        break;
1046    while ( 1 )
1047    {
1048        *a2 = v8 + 1;
1049        if ( *a2 == 0 )
1050            *a2 = 0xFF;
1051        a2++;
1052    }
1053    v8 = *a2;
1054    v9 = *a2 & 0x3F;
1055    if ( v9 == 56 )
1056        break;
1057    while ( 1 )
1058    {
1059        *a2 = v8 + 1;
1060        if ( *a2 == 0 )
1061            *a2 = 0xFF;
1062        a2++;
1063    }
1064    v8 = *a2;
1065    v9 = *a2 & 0x3F;
1066    if ( v9 == 56 )
1067        break;
1068    while ( 1 )
1069    {
1070        *a2 = v8 + 1;
1071        if ( *a2 == 0 )
1072            *a2 = 0xFF;
1073        a2++;
1074    }
1075    v8 = *a2;
1076    v9 = *a2 & 0x3F;
1077    if ( v9 == 56 )
1078        break;
1079    while ( 1 )
1080    {
1081        *a2
```

With this, we can begin with emulating the custom Virtual Machine of the program, here’s a snippet for the first few opcodes of the VM:

```
18 class VM:
19     def __init__(self, mem_bytes):
20         self.regs = [1, 2, 3, 4]
21         self.mem = bytearray(mem_bytes)
22
23     def runShell(self, opc):
24         if opc == 0xCA:
25             v4 = (self.regs[0] + self.regs[1]) & 0xFFFFFFFF
26             v6 = (self.regs[2] - self.regs[3]) & 0xFFFFFFFF
27             self.regs[0] = (v6 ^ v4) & 0xFFFFFFFF
28
29         elif opc == 0xFE:
30             r = self.regs[1] & 31
31             self.regs[0] = rol32(self.regs[0], r)
32
33         elif opc == 0xA7:
34             self.regs[2] = (self.regs[0] - (0x9E3779B1 * (self.regs[2] & 0xFFFFFFFF))) & 0xFFFFFFFF
35
36         elif opc == 0xD3:
37             buf = b''.join(struct.pack("<I", r) for r in self.regs)
38             dig = hashlib.sha1(buf).digest()
39             self.regs[3] = struct.unpack("<I", dig[:4])[0]
```

Outside of the VM bytecode map, in the `initMaskedShell` function, we can see the value `16777619` being constantly XOR’d with a bunch of bytes:

```
33     v6 = *(_DWORD *)(v3 + 8);
34     v7 = *(_DWORD *)(v3 + 12);
35     v8 = HIBYTE(v7)
36     ^ (16777619
37     * (BYTE2(v7)
38     ^ (16777619
39     * (BYTE1(v7)
40     ^ (16777619
41     * ((unsigned __int8)v7
42     ^ (16777619
43     * (HIBYTE(v6)
44     ^ (16777619
45     * (BYTE2(v6)
46     ^ (16777619
47     * (BYTE1(v6)
48     ^ (16777619
49     * ((unsigned __int8)v6
50     ^ (16777619
51     * (HIBYTE(*(_DWORD *)(v3 + 4))
52     ^ (16777619
53     * ((unsigned __int8)*(_DWORD *)(v3 + 4))
```

At some point of the program, one may also see a constant value of `0x811c9dc5` being loaded into a register. With simple research and searching, we can identify this being the 32-bit hashing function called FNV1:

FNV parameters ^{[4][14]}			
Size in bits $n = 2^s$	Representation	FNV prime	FNV offset basis
32	Expression	$2^{24} + 2^8 + 0x93$	
	Decimal	16777619	2166136261
	Hexadecimal	0x01000193	0x811c9dc5

We can implement this in python as such:

```
...  
  
def fnv1(data):  
    val = 0x811c9dc5  
    for c in data:  
        val = ((0x1000193 * val) ^ c) & 0xffffffff  
    return val  
  
...
```

Along with this, there's a custom Rolling Hash function which can be spotted:

```
72     while ( 1 )  
73     {  
74         v13 = a1 + v4;  
75         if ( __OFADD__(a1, v4) )  
76             return (_BYTE *)raiseOverflow(a1);  
77         if ( v13 < 0 || v13 >= v1 )  
78             return (_BYTE *)raiseIndexError2(a1 + v4, v1 - 1);  
79         ++a1;  
80         v14 = (unsigned __int8)(*( _BYTE *) (v10 + v13 + 8) ^ v11);  
81         v8 = v14 ^ (16777619 * v8);  
82         v12 = ROL4(v14 ^ v12, 3);  
83         v11 = v8;
```

This is also a 32-bit hashing function which we can implement in python as such:

```
...  
  
def rol32(x, r):  
  
    r &= 31  
  
    return ((x << r) | (x >> (32 - r))) & 0xFFFFFFFF  
  
...
```

Before we emulate the VM, we still need to find what the VM is actually deriving. So we'll need to find out how that AES key and iv is being derived, as well as the encryption mode.

First we'll head to the `deriveNeuralKeyPair` function to see what we can find:

```
390 if ( *address )
391     goto LABEL_93;
392 deriveNeuralKeyPair__66lack83hell_u3189(v77, &v75);
393 v18 = v66;

50 v11 = _mm_unpacklo_epi64(
51     (__m128i)((unsigned __int8)v7
52         | ((BYTE1(v7)
53             | ((v9
54                 | ((v8
55                     | (((unsigned __int8)v5
56                         | ((BYTE1(v5) | ((BYTE2(v5) | ((unsigned __int64)HIBYTE(v5) << 8)) << 8)) << 8)) << 8)) << 8)),
57         (__m128i)((unsigned __int8)v10
58             | ((BYTE1(v10)
59                 | ((BYTE2(v10)
60                     | ((HIBYTE(v10)
61                         | (((unsigned __int8)v6 | ((BYTE2(v6) | ((unsigned __int64)HIBYTE(v6) << 8)) << 16)) << 8)) << 8)) << 8))),
62     result = (_BYTE *)_emutls_get_address(refptr__emutls_v_nimInErrorMode__system_u4398);

95 if ( !*v13 )
96 {
97     newSeq__stdZsysrand_u60(&v25, 16);
98     result = (_BYTE *)v25;
99     if ( v25 <= 0 )
100     {
101         v20 = 0;
102     }
103     else
104     {
105         v26[8] = v29[0];
106         if ( result == (_BYTE *)1 )
107         {
108             v20 = 1;
109         }
110         else
111         {
112             v26[9] = BYTE1(v29[0]);
113             if ( result == (_BYTE *)2 )
114             {
115                 v20 = 2;
116             }
117             else
118             {
119                 v26[10] = BYTE2(v29[0]);
120                 if ( result == (_BYTE *)3 )
121                 {
122                     v20 = 3;
123                 }
124             }
125         }
126     }
127 }
```

Again, it's very messy, but looking properly we can see four 32-bit registers being loaded first:

```
v7 = *a1;    // a1[0]
```

```
v5 = a1[1];  // a1[1]
```

```
v10 = a1[2]; // a1[2]
```

```
v6 = a1[3];  // a1[3]
```

This is equivalent to AX, BC, CX, DX in the VM.

Then, there is an obvious long `\_mm\_unpacklo\_epi64` which packing a bunch of bytes and stored into `v30[5]` = v11.

```
result = (_BYTE *)finish__66lack83hell_u3284(v30, &v27);
if ( !*v13 )
{
    newSeq__stdZsysrand_u60(&v23, 32);
    v14 = v23;
    for ( i = 0; i != 32; ++i )
    {
        if ( v14 <= i )
            return (_BYTE *)raiseIndexError2(i, v14 - 1);
        *(_BYTE *) (v24 + i + 8) = v27.m128i_i8[i];
    }
}
```

It then allocates 32 bytes into the buffer `v23` which tells us this is the AES key. After that, right below the AES key computation, we can see the IV being computed as well:

```
if ( v16 )
{
    v17 = _mm_loadu_si128(&v27);
    v18 = _mm_loadu_si128(&v28);
    memset(v29, 0, sizeof(v29));
    LODWORD(v31[0]) = 32;
    *(__m128i *)&v31[5] = v17;
    *(__m128i *)&v31[7] = v18;
    result = (_BYTE *)finish__66lack83hell_u3284(v31, v29);
    if ( !*v13 )
    {
        newSeq__stdZsysrand_u60(&v25, 16);
        result = (_BYTE *)v25;
        if ( v25 <= 0 )
        {
            v20 = 0;
        }
    }
}
```

If we translate all this into readable pseudocode, we get this:

```
uint8_t seed[16] = LE(a1[0]) || LE(a1[1]) || LE(a1[2]) || LE(a1[3]);
```

```
uint8_t key[32] = SHA256(seed);
```

```
uint8_t iv_full[32] = SHA256(key);
```

```
uint8_t iv[16] = iv_full[0..15];
```

From all this, we can figure out that by using the VM, we'll need to dump the final four 32-bit registers from the VM to retrieve the AES key.

By running the emulator, we're able to retrieve the following registers:

`0xd55940cd`, `0x3178961e`, `0xfc5b0e25`, `0x4c913b7e`

To compute the AES key, we're able to utilize python's hashlib library as such:

```
1 import hashlib
2 import struct
3
4 def deriveKey(regs):
5     seed = b''.join(struct.pack("<I", r & 0xFFFFFFFF) for r in regs)
6
7     # SHA256 seed = AES key
8     key = hashlib.sha256(seed).digest()
9
10    return key
11
12 if __name__ == "__main__":
13     regs = [0x3e7f6dfc, 0xd1c183d4, 0xc5d97911, 0x64e1bde2] # Found in
14
15     key = deriveKey(regs)
16
17     print("[Key computed from final registers:]")
18     print("Key:", key.hex())
19
```

```
PS C:\Users\jyber\OneDrive\Desktop> python3 C:\Users\jyber\OneDrive\Desktop\aeskey.py
Key computed from final registers:
Key: 0x00000000000000000000000000000000
PS C:\Users\jyber\OneDrive\Desktop>
```

Now what we're left is the IV and Ciphertext for AES-CTR decryption.

Heading back to the `executeAlgorithmicEnigmaSystem` function, we can see what is most likely the AES encryption function, which takes 3 input buffers as shown:

```
494 v38 = isPngFile__66lack83hell_u201(&v72);
495 if ( *address
496     || v38
497     && (v39 = __mm_loadu_si128(&v74),
498         v71 = v46,
499         v72 = (unsigned __int128)v39,
500         v70 = (unsigned __int128)v47,
501         executeAlgorithmicEnigmaSystem__66lack83hell_u3698(&v72, &v71, &v70),
502         *address) )
503 {
504
505 LABEL_31:
506     qmemcpy(v53, v54, 0x3C8u);
507     encrypt__66lack83hell_u4416((unsigned int)v53, (unsigned int)&v54[121], 32, (unsigned int)&v54[125], 32);
508     if ( *address )
509         goto LABEL_29;
510     inc128__pkgZnimcryptoZbcmode_u176(&v54[121], 32);
511     if ( *address )
512         goto LABEL_29;
513     v18 = 0;
514     if ( v19 >= v16 )
515         goto LABEL_27;
516 }
517 LABEL_66:
518
```

Here, within the function, with the help of Nim module's source code, libraries and a debugger, we should be able to find out that the image is encrypted with AES-CTR.

Finally, scrolling a little bit more, we can see how the IV is being stored:

\ \ \

```
*(DWORD*)(v22 + 8) = 1213153838; // 1213153838 = 0x484F422E = .BOH ascii (header of png)
```

```
rawNewString(&v46, v23.m128i_i64[0] + 4);
```

...

```
add__66lack83hell_u396(&v47, v22 + 8, 4); // Appends 4 byte .BOH header into the .png
```

```
add__66lack83hell_u396(&v47, v43, v40); // Appends IV into first 16 bytes (after .BOH) of the .png
```

```
add__66lack83hell_u396(&v47, v37, v16); // Appends cipher text bytes into .png
```

\ \ \

Now that we know that the IV is stored as the first 16 bytes (after the .BOH signature) in the encrypted image, as well as the ciphertext, we can craft a final script to decrypt our MasterKey.png with the help of Python's Crypto library:

```
1 import os, binascii
2 from Crypto.Cipher import AES
3
4 KEY = bytes.fromhex(
5     "5ef0d739b060bf9d9faeb15f9e7834780f077805b7d71c07de188498f8d19fd1" # Predefined AES key de
6 )
7
8 PNG_MAGIC = b"\x89PNG\r\n\x1a\n"
9
10 def fileDecrypt(path: str, key: bytes, force_save: bool = False):
11     with open(path, "rb") as f:
12         blob = f.read()
13
14     if not blob.startswith(b".BOH"): # Looks for files with BOH signature header
15         print(f"[!] {path}: missing .BOH signature, skipping")
16         return
17
18     iv = blob[4:20] # IV derived from first 16 bytes AFTER skipping the .BOH header
19     ct = blob[20:] # Rest of ciphertext of encrypted file (AES-CTR mode)
20
21     # Try BIG-endian first
22     candidates = [
23         ("big", endianDecrypt(iv, ct, key, "big")),
24         ("little", endianDecrypt(iv, ct, key, "little")),
25     ]
26
27     chosen = None
28     for endian, plain in candidates:
29         print(f"[+] {path}: endian={endian} first8={hex8(plain)}")
30         if png(plain):
31             print(f"[+] {path}: valid PNG signature detected with {endian}-endian IV")
32             chosen = plain
33             break
```


After running the script, we're able to retrieve our flag:

```
[*] MasterKey.png.BoH: endian=big first8=89504e470d0a1a0a  
[+] MasterKey.png.BoH: valid PNG signature detected with big-endian IV  
Decrypted: MasterKey.png.dec.png
```

